



DCC
DEPARTAMENTO DE CIENCIA
DE LA COMPUTACIÓN

IIC2343

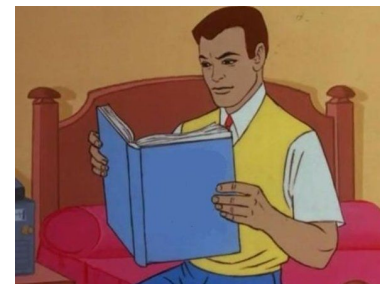
Arquitectura de Computadores

Clase 3 - Almacenamiento de Datos

Profesor: Germán Leandro Contreras Sagredo

Bibliografía

- Apuntes históricos. Hans Löbel, Alejandro Echeverría
 - 3 - Almacenamiento de datos
- **D. Patterson, Computer Organization and Design RISC-V Edition: The Hardware Software Interface. Morgan Kaufmann, 2020.**
 - Capítulos A.7-A.8-A.9. Página A-47, 792 en PDF.



Objetivos de la clase

- Entender la necesidad de almacenar datos en nuestra máquina programable.
- Conocer distintos circuitos digitales y componentes que permiten almacenar datos.
- Realizar ejercicios que consoliden los conocimientos anteriores.

Hasta ahora...

Ya tenemos el mecanismo y los medios para poder operar con números en nuestra máquina programable.

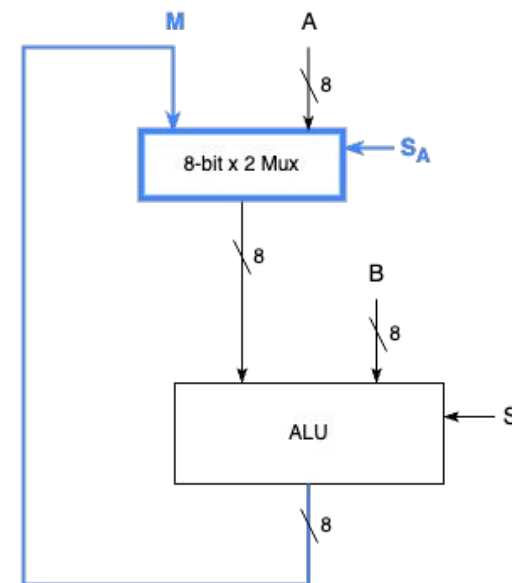
Pero, ¿qué podemos hacer con nuestra ALU actualmente?

- Realizar 8 operaciones distintas entre dos números A y B .
- Observar el resultado.
¿Podemos hacer algo con el resultado?



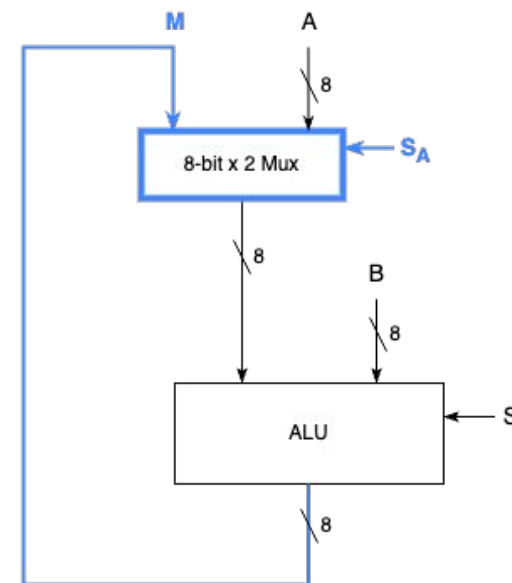
Creando una calculadora con la ALU

- Queremos usar el resultado de nuestra ALU. Una primera opción es conectar el resultado con la **entrada A** de la unidad mediante un Mux de dos entradas con un nuevo bit de selección para decidir qué valor operar.
- ¿Existe algún problema con esto?
 - ¿Cuánto tiempo toma la operación?
 - ¿Qué tan rápido debo cambiar la señal S_A ?
 - ¿Cuándo o cómo me detengo?
 - ¿Cómo retengo el resultado final?



Creando una calculadora con la ALU

- Esta alternativa no es óptima. Requerimos de un componente que permita:
 - Almacenar un estado (bit o secuencia de bits).
 - Realizar el cambio de estado en **un instante determinado**.
- Los componentes que necesitamos son los **latches** y **flip-flops**.

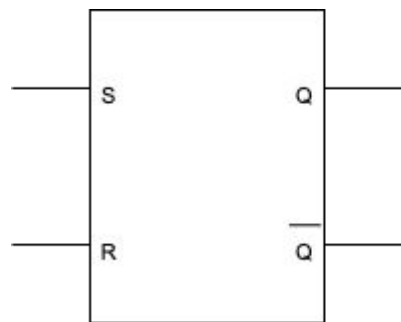


Latch

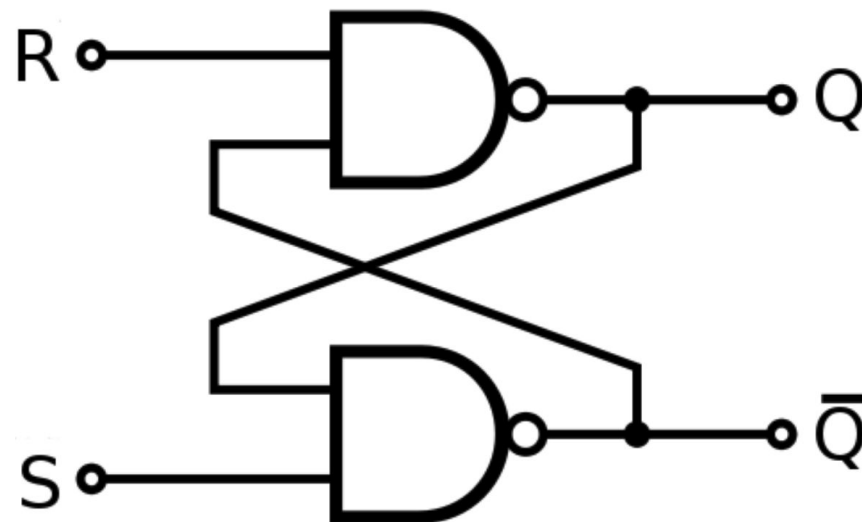
- Componente que permite **guardar el estado anterior de una señal**. El término proviene del verbo en inglés: “encerrar” (con pestillo). Estos se conocen como **circuitos secuenciales**.
- Se construyen con compuertas **que ya conocemos** (NAND, NOR, NOT, etc.).
- Existen dos tipos principalmente: RS, D.
- Distintas combinaciones de estos nos permitirán crear distintos tipos de memoria.

Latch RS

Componente que realiza *set* ($Q = 1$), *reset* ($Q = 0$) o mantiene un estado previo: $Q(t+1) = Q(t)$. A continuación, veremos cómo funciona.



Abstracción



S	R	$Q(t+1)$
0	0	-
0	1	0
1	0	1
1	1	$Q(t)$

* Asuma que $Q(t)$ es el "instante previo" y $Q(t+1)$ es el "instante actual".

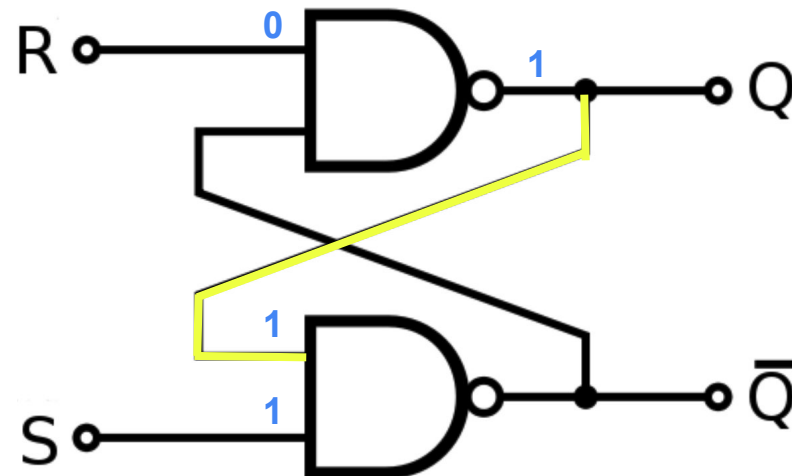
Latch RS - Caso a caso

$$R = 0, S = 1$$

Partamos el flujo por R . Como su valor es 0, sabemos que la compuerta NAND entregará como salida 1.

Luego, como la compuerta NAND inferior recibe dos *input* 1, su salida será un 0.

De esta forma, $Q = 1$ y $\neg Q = 0$.



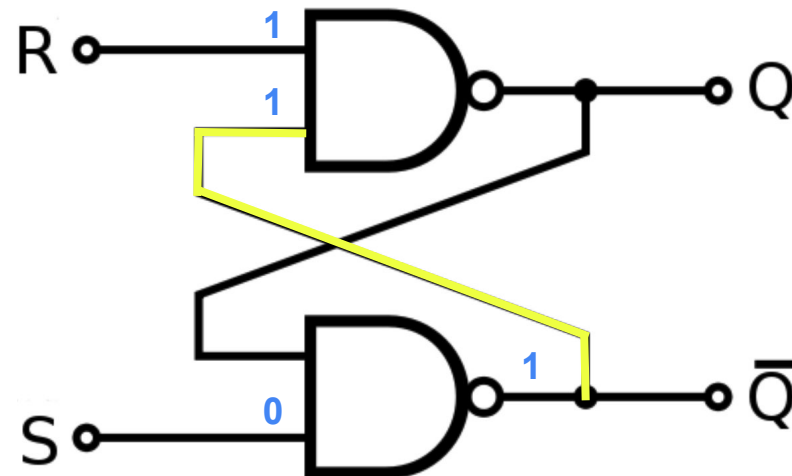
Latch RS - Caso a caso

$$R = 1, S = 0$$

Partamos el flujo por S. Como su valor es 0, sabemos que la compuerta NAND entregará como salida 1.

Luego, como la compuerta NAND superior recibe dos *input* 1, su salida será un 0.

De esta forma, $Q = 0$ y $\neg Q = 1$.

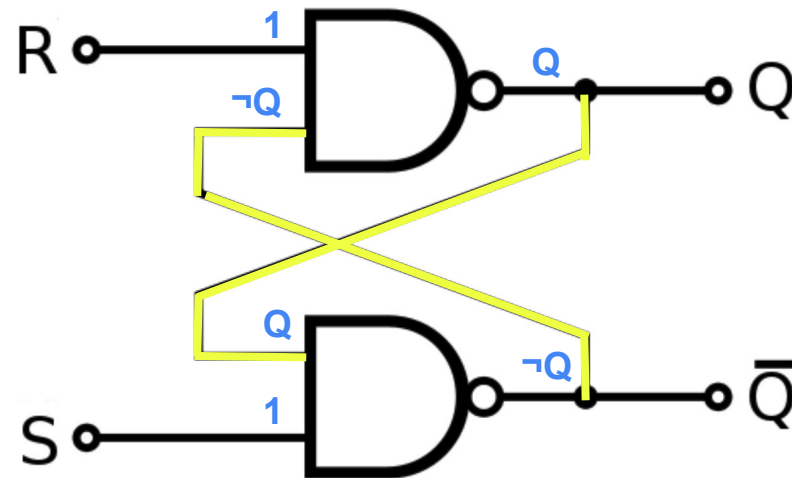


Latch RS - Caso a caso

$$R = 1, S = 1$$

Independiente del flujo que sigamos, vemos que ambas salidas dependen de Q y $\neg Q$ por leyes de dominación.

A continuación, realizaremos las operaciones lógicas para verificar la permanencia de los estados.



Latch RS - Caso a caso

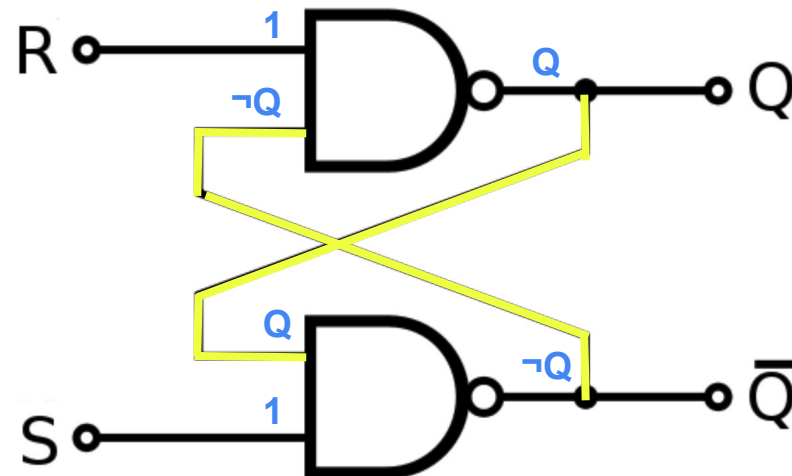
$$R = 1, S = 1$$

Por dominación.

$$\begin{aligned} R \text{ NAND } \neg Q \\ &= \text{NOT}(1 \text{ AND } \neg Q) \\ &= \text{NOT}(\neg Q) = Q \end{aligned}$$

$$\begin{aligned} S \text{ NAND } Q \\ &= \text{NOT}(1 \text{ AND } Q) \\ &= \text{NOT}(Q) = \neg Q \end{aligned}$$

¡Retenemos el estado!



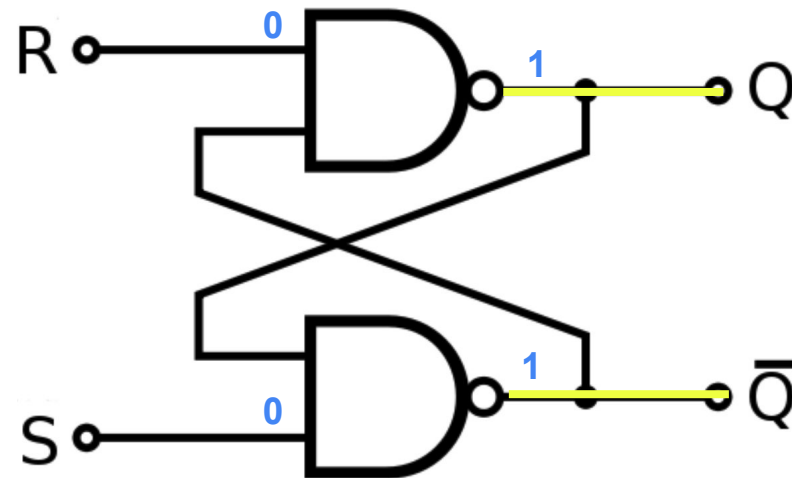
Latch RS - Caso a caso

$$R = 0, S = 0$$

En ambos flujos, la salida de la compuerta NAND será 1.

De esta forma, $Q = \neg Q = 1$.

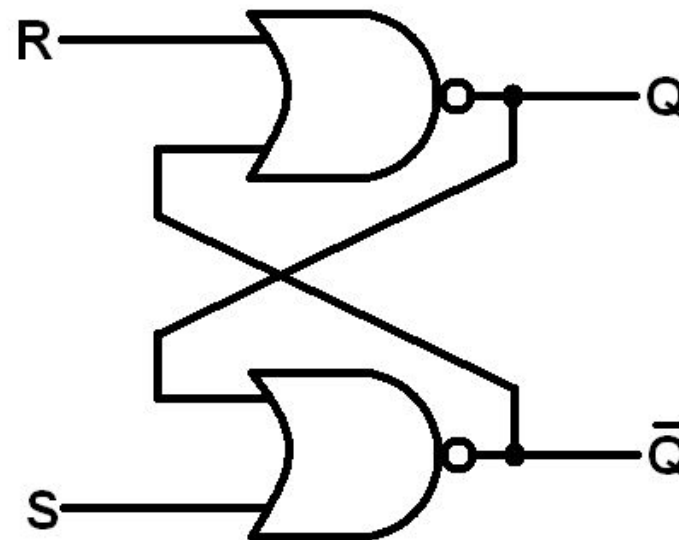
Esto se considera un estado **inválido**, ya que no se respeta el inverso del estado final.



Latch RS - Otra construcción

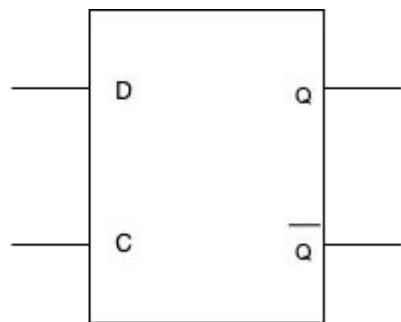
Recordando que tanto la compuerta NAND como la compuerta NOR se consideran universales, también existe la construcción del latch RS con esta última.

Por temas de comodidad y consistencia, seguiremos utilizando compuertas NAND para el resto de componentes a construir.

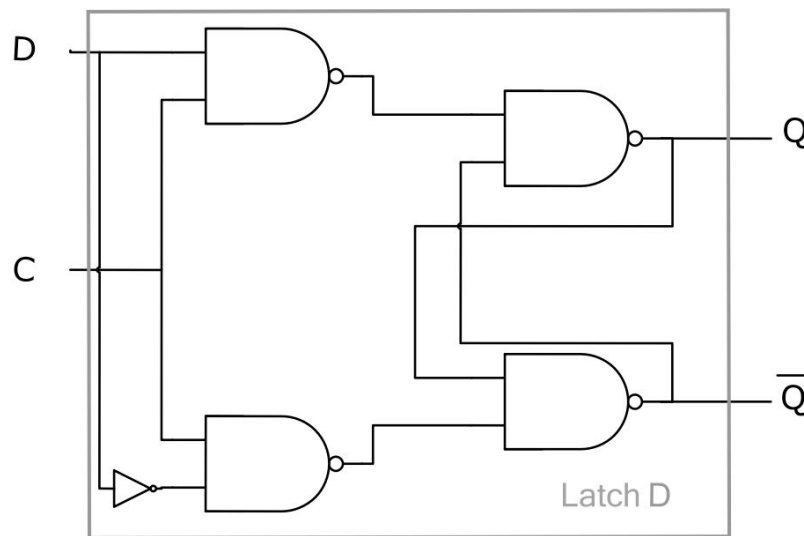


Latch D

Componente que actualiza el valor de un estado Q con una señal D si, y solo si la señal de control está activa ($C = 1$). $D = \text{Data}$.



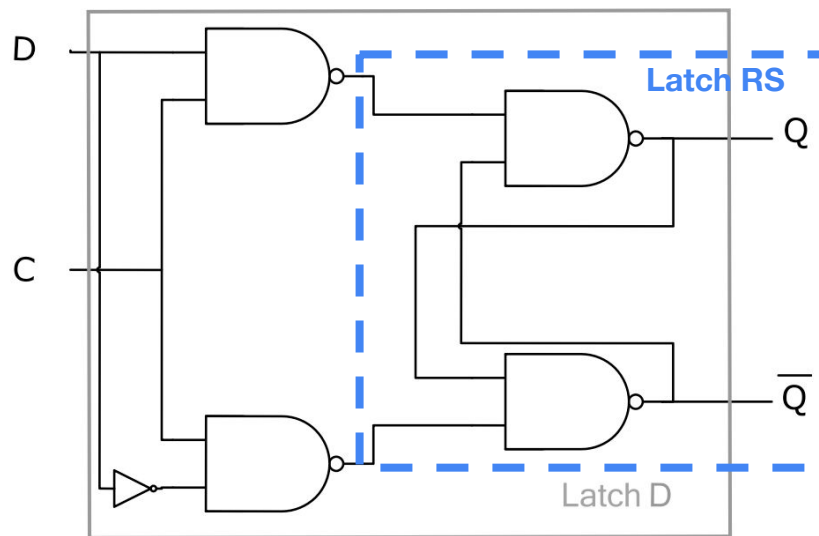
Abstracción



C	D	$Q(t+1)$
0	0	$Q(t)$
0	1	$Q(t)$
1	0	0
1	1	1

Latch D

Si observamos bien, es una versión del latch RS **sin estados inválidos**.

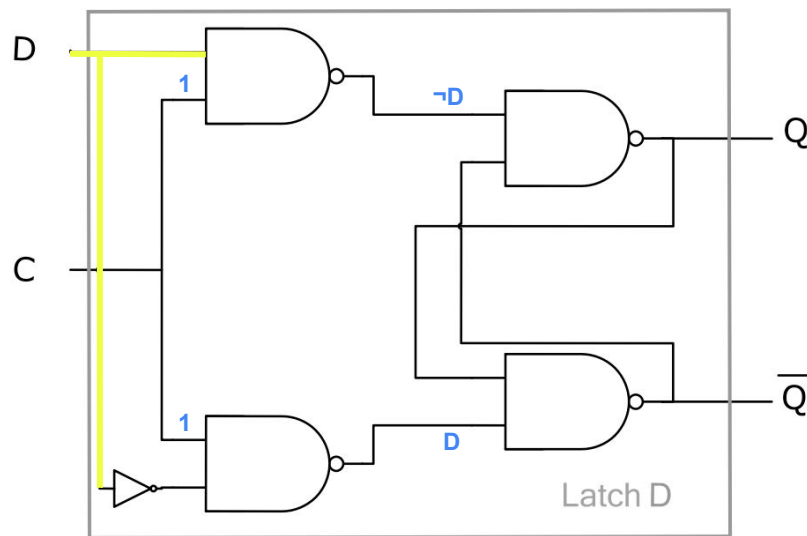


Si $C = 0$, las dos entradas del latch RS “interno” se vuelven 1, siendo esta la combinación que **retiene el estado**.

Por otra parte, dada la negación de D en la secuencia inferior, nunca se tendrá la combinación inválida del latch RS.

Latch D

¿Y si $C = 1$?



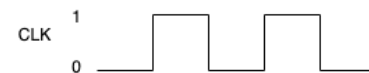
Las salidas de ambas compuertas NAND de entrada serán el inverso de la señal ingresada. Luego, como una de las señales D o $\neg D$ será 0, una de las compuertas NAND de salida tendrá valor 1, lo que permite el cambio de estado del componente.

Latch D

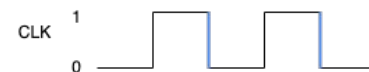
- Al ser su manejo de señales más intuitivo, lo utilizaremos por sobre el latch RS.
- Ahora, ¿es este el componente que necesitamos? Podemos almacenar el valor de entrada con la señal de control, no obstante, si la dejamos activa el estado final del componente podría seguir cambiando si cambia el dato de ingreso.
- **Necesitamos que la actualización ocurra una única vez en un instante dado.**

Flanco de subida y de bajada

- Para poder actualizar un estado en un instante dado, necesitamos entender el concepto de flanco.
- Supongamos que tenemos una señal CLK que cambia su valor de 0 a 1 y de 1 a 0 con una frecuencia constante.
- El flanco de subida corresponderá al instante en que la señal cambia de 0 a 1, mientras que el flanco de bajada será el instante en que la señal cambia de 1 a 0.



Flanco de
subida



Flanco de
bajada

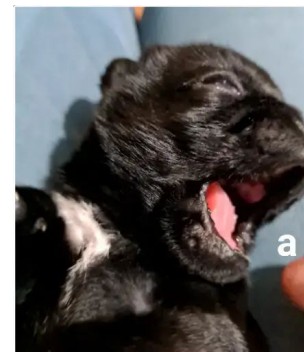
Flanco de subida y de bajada

Queremos, entonces, que nuestro componente de almacenamiento actualice su estado si, y solo si **existe un flanco de subida en la señal de control**.

Así es como se construye el componente **flip-flop**.

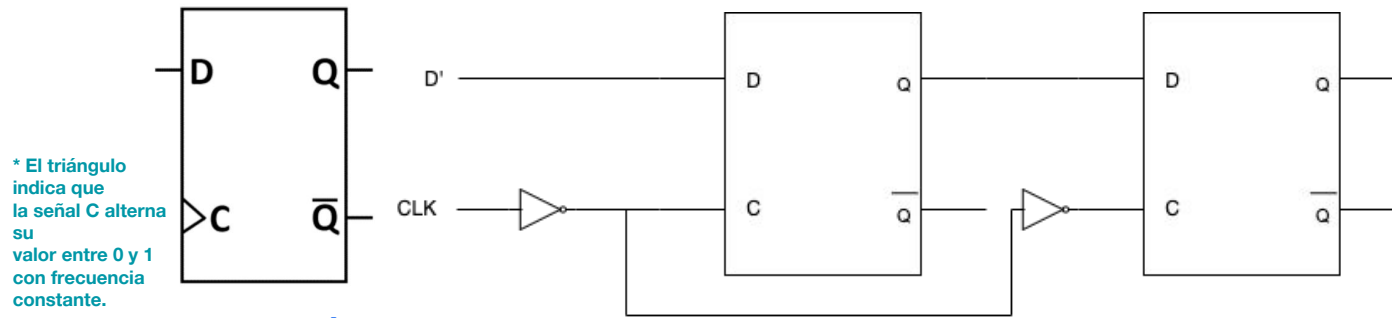


Su nombre no viene del término para “sandalias” en inglés, sino que su uso data de los 1900s, haciendo referencia a la “inversión completa de dirección”.



Flip-flop D

- Componente que permite **guardar el estado anterior de una señal en un instante dado**.
- El instante de almacenamiento corresponde al flanco de subida de la señal de control. Veremos a continuación el caso a caso.



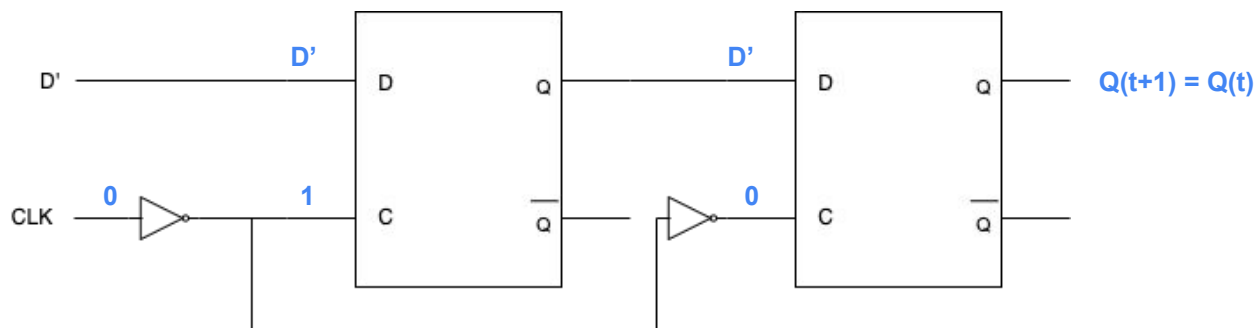
Abstracción

CLK	D	$Q(t+1)$
0/1/↓	0/1	$Q(t)$
↑	0	0
↑	1	1

Flip-flop D - Caso a caso

CLK = 0

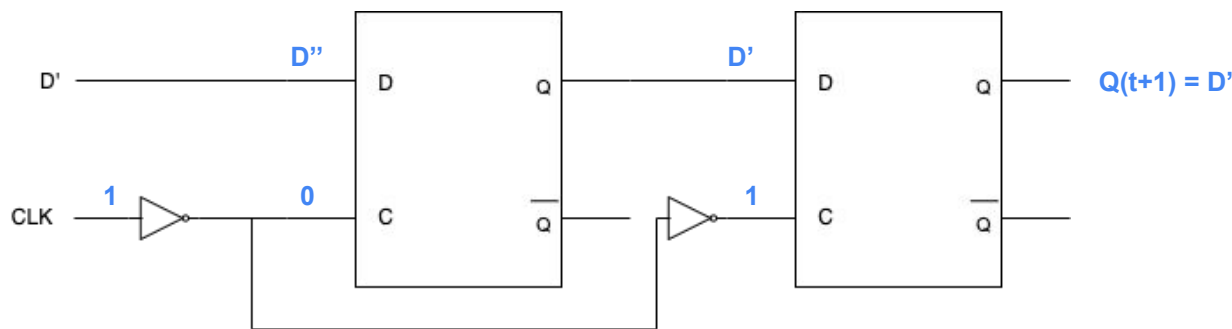
Se habilita el almacenamiento en el primer latch pero no en el segundo, la salida del segundo latch (estado final) sigue siendo el estado anterior.



Flip-flop D - Caso a caso

$\text{CLK} = 0 \rightarrow \text{CLK} = 1$

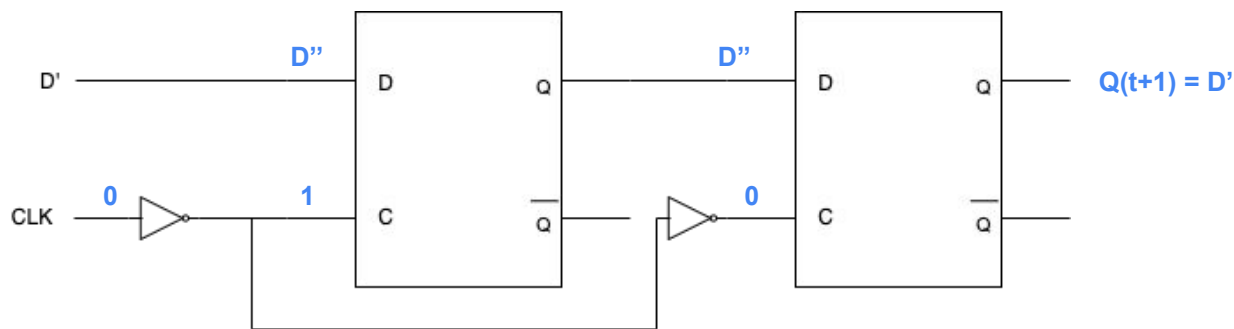
Se habilita el almacenamiento en el segundo latch pero no en el primero, por lo que aunque ingresara un nuevo valor D'' este no se almacenará preliminarmente. Por otra parte, el estado final ahora se convierte en D' .



Flip-flop D - Caso a caso

$\text{CLK} = 1 \rightarrow \text{CLK} = 0$

Se vuelve a habilitar el almacenamiento en el primer latch y se bloquea el segundo. Esto hace que ahora la nueva señal D'' sea almacenada en el primer latch **sin alterar nuestro estado final**. Así termina un ciclo de la señal CLK.



Flip-flop D

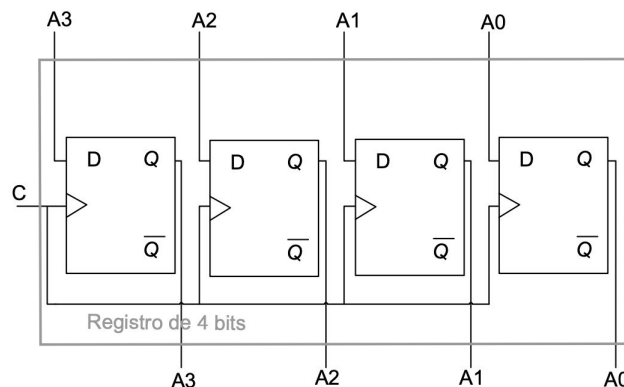
- Los flip-flop D (construidos con latches D) serán los componentes a utilizar para crear los registros de nuestra máquina programable.
- Cabe señalar, no obstante, la existencia de otros flip-flop):
 - Flip-flop RS
 - Flip-flop JK
 - Flip-flop T

Más información en los anexos

Registro de memoria

Volviendo al flip-flop D: Si deseamos almacenar números de 4 bits, por ejemplo, simplemente creamos un componente con 4 flip-flops D y una única señal C que alterna su valor con cierta frecuencia.

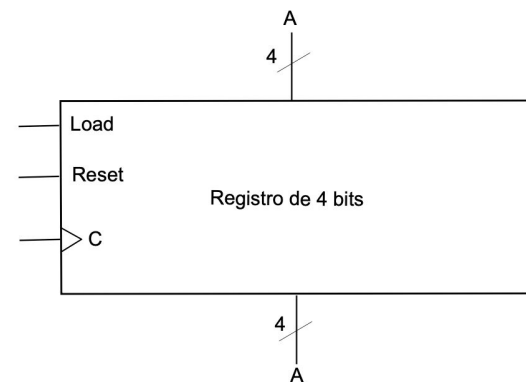
Esto corresponde a un **registro de memoria de 4 bits**.



Registro de memoria

Ahora, si lo dejamos así como está, el valor almacenado cambiará siempre que exista un flanco de subida. Queremos que esto ocurra **solo cuando deseemos hacerlo**. Para esto, agregamos dos señales: Una para cargar o no el valor de entrada (**Load**) y otra para limpiar el valor almacenado (**Reset**).

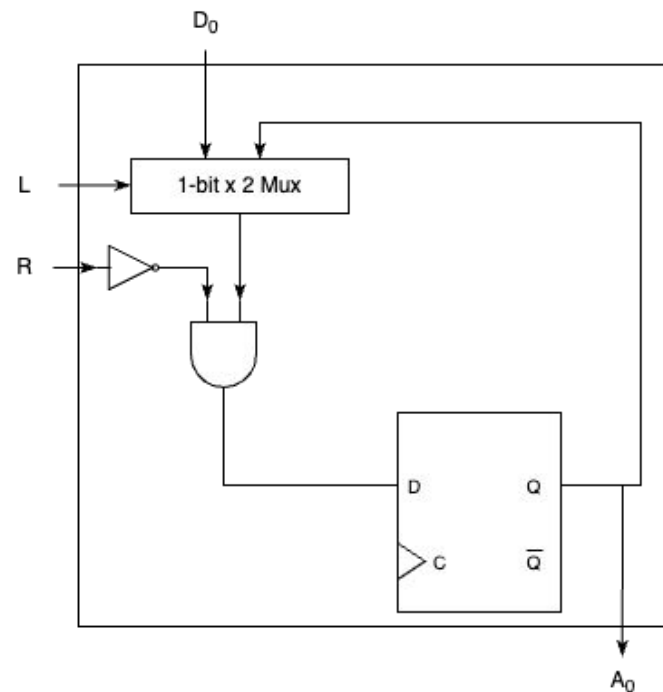
¿Cómo se modifica el diagrama interno del registro de 4 bits para que utilice las señales *Load* y *Reset*?



Registro de memoria - Señales *Load* (**L**) y *Reset* (**R**)

Se muestra el diagrama interno de las señales *Load* (**L**) y *Reset* (**R**) conectadas con el bit menos significativo del registro (**A₀**). Se puede extrapolar para el resto de bits utilizando las mismas señales *L* y *R*.

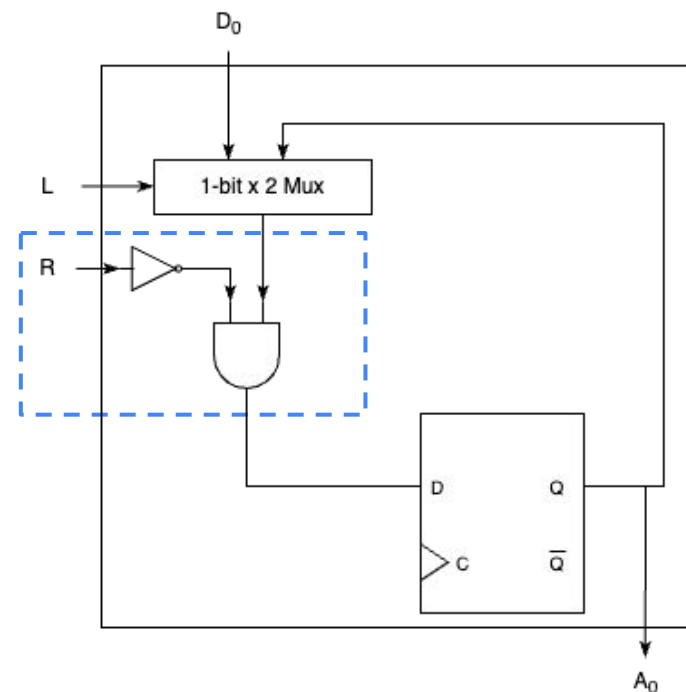
A continuación, se explicará cómo interactúa cada una de ellas.



Registro de memoria - Señales *Load* y *Reset*

Reset

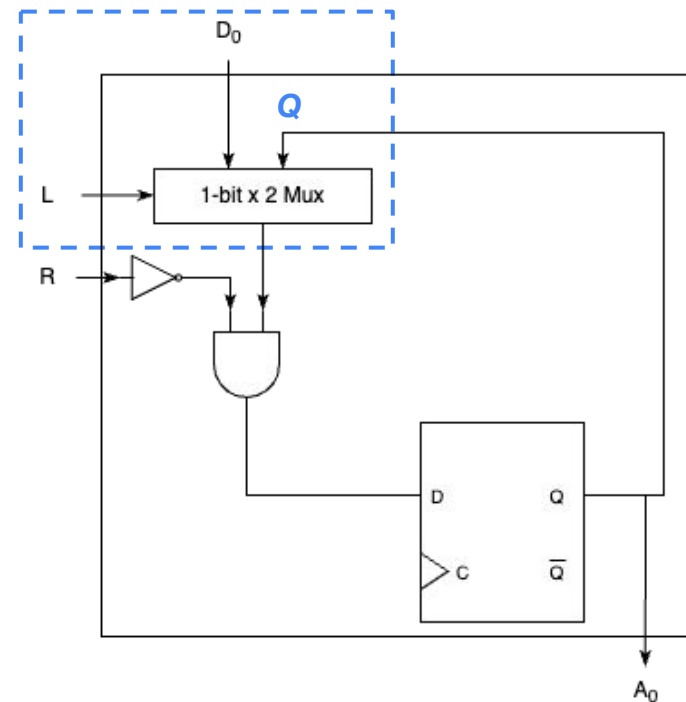
Esta señal actúa directamente sobre la entrada D del flip-flop donde se almacena el bit menos significativo del registro. Si $R = 1$, el flip-flop guardará un 0 al ser el valor resultante de la compuerta AND. Si $R = 0$, el resultado a almacenar será la otra señal de la compuerta AND, que veremos a continuación.



Registro de memoria - Señales *Load* y *Reset*

Set

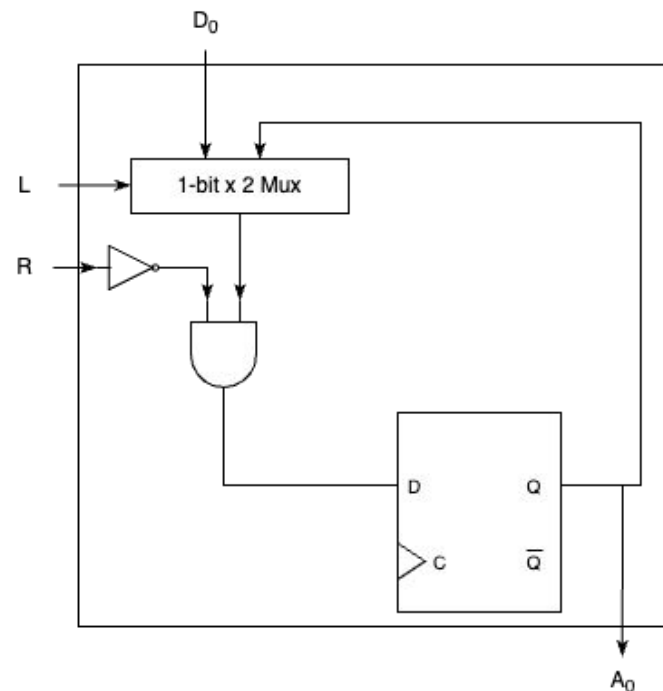
Esta señal se utiliza como el selector de un multiplexor que escoge entre una señal de entrada D_0 y el estado original Q del flip-flop. Si $L = 0$, se selecciona el estado Q y se almacena (*i.e.* se sobrescribe con el mismo valor original). Si $L = 1$, se escoge la señal de entrada D_0 y se escribe en el flip-flop, por lo que $A_0 = D_0$.



Registro de memoria - Señales *Load* y *Reset*

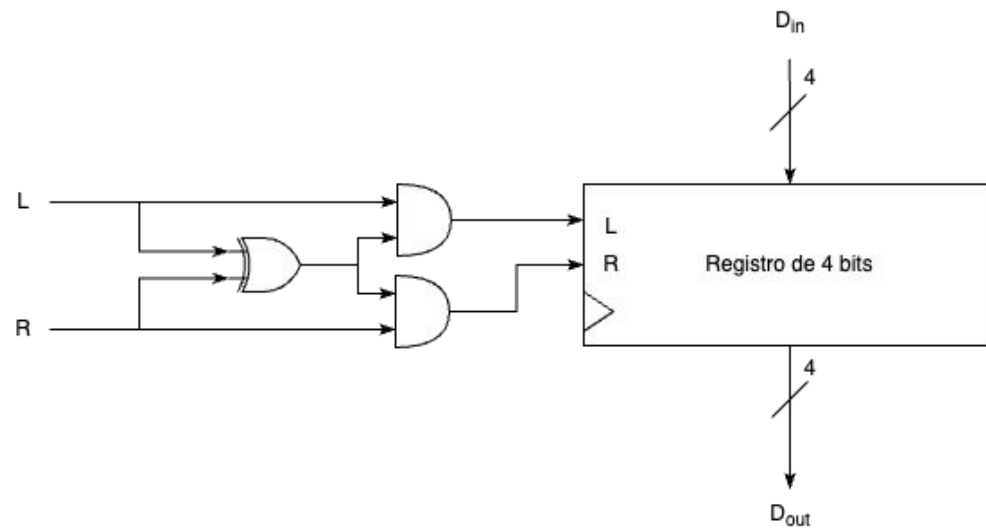
Esta construcción tiene una consecuencia relevante: Si $L = R = 1$, por el orden de las compuertas la señal R tendrá prioridad por sobre la señal L y el flip-flop terminará almacenando un 0.

¿Se puede modificar el diagrama de forma que si $L = R = 1$, el flip-flop **no cambie su valor**?



Registro de memoria - Señales *Load* y *Reset*

Antes de ingresar al registro, se pueden conectar ambas señales a una compuerta XOR. Esto resulta en que si son iguales (ya sean 0 o 1), el registro siempre recibirá como entradas $L = R = 0$, lo que asegura que su valor no se actualice.

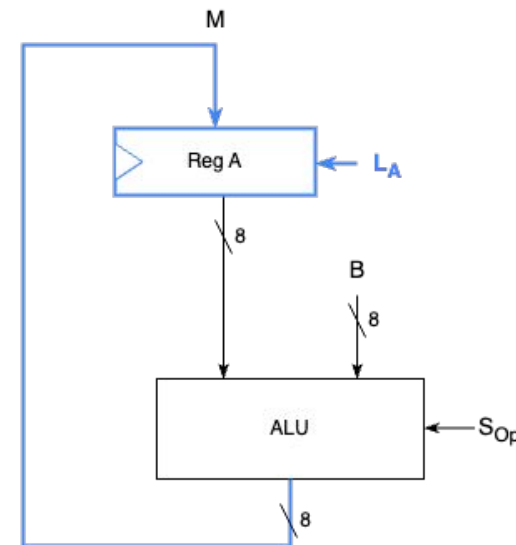


* Esta modificación también podría estar dentro del componente, este es un ejemplo.

Creando una calculadora con la ALU

Ahora que contamos con un registro de memoria que puede almacenar estados y cambiarlos en un instante dado, podemos tener una solución óptima para nuestra calculadora.

Con la señal L_A (*Load A*), controlamos la sobrescritura o no del registro de A con el resultado de la ALU.

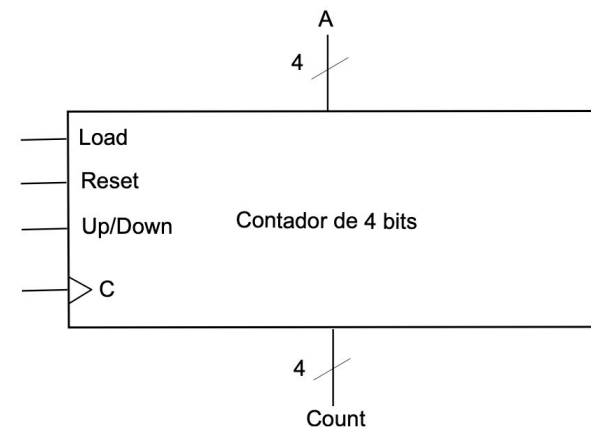


Si bien ya tenemos una primera calculadora, debemos conocer otros tipos de memoria a utilizar en nuestra máquina.

Contador

Podemos convertir nuestro registro de memoria en un **contador**, añadiendo señales que incrementan (**Up**) o decrementan (**Down**) el valor almacenado.

Ejercicio: Modifique el diagrama interno del registro de memoria con *Load* y *Reset* de 4 bits para que incluya las señales *Up* y *Down*.



Memorias

Si bien los registros nos permiten almacenar unidades de información o **palabras** (cadena finita de bits), queremos también un componente en el que podamos acceder o modificar distintos datos según una **dirección de memoria**.

Este proceso de acceso (ya sea de lectura o escritura) se conoce como **direccionamiento**.

Dirección en decimal	Dirección en binario	Palabra
0	000	<i>Palabra₀</i>
1	001	<i>Palabra₁</i>
2	010	<i>Palabra₂</i>
3	011	<i>Palabra₃</i>
4	100	<i>Palabra₄</i>
5	101	<i>Palabra₅</i>
6	110	<i>Palabra₆</i>
7	111	<i>Palabra₇</i>

Tipos de memoria

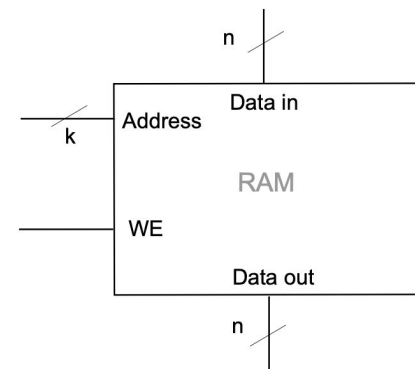
En nuestra máquina programable, usaremos principalmente dos tipos de memoria:

- Memoria de lectura y escritura: *Random access memory* o **RAM**.
- Memoria de solo lectura: *Read only memory* o **ROM**.

Tipos de memoria - RAM

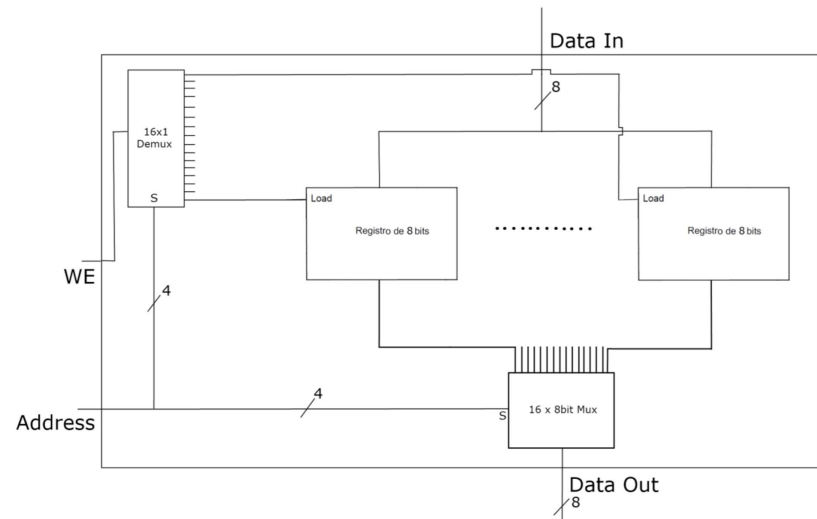
Extensión de los registros. Ocupan flip-flops pero extienden sus señales de control para poder acceder a **una sola palabra** y habilitar o no la escritura de ella. Se les llama *random access memory* porque el acceso a cualquier dirección de memoria es directo (no se deben recorrer las direcciones anteriores).

Con k bits para el bus de direccionamiento y buses de entrada/salida de n bits, la RAM almacena 2^k palabras de memoria de n bits. Por otra parte, tenemos una señal de un bit WE (**W**rite **E**nable) que habilita o no la escritura sobre la palabra seleccionada.



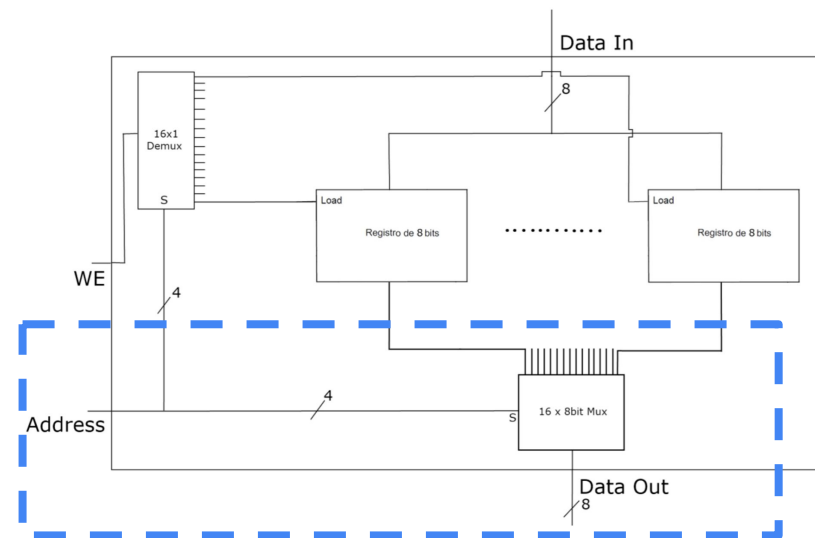
Tipos de memoria - RAM

Haciendo uso solo de los componentes vistos en clases hasta ahora, podemos construir una RAM de palabras de 8 bits (1 byte) y 16 direcciones de memoria de la siguiente manera.



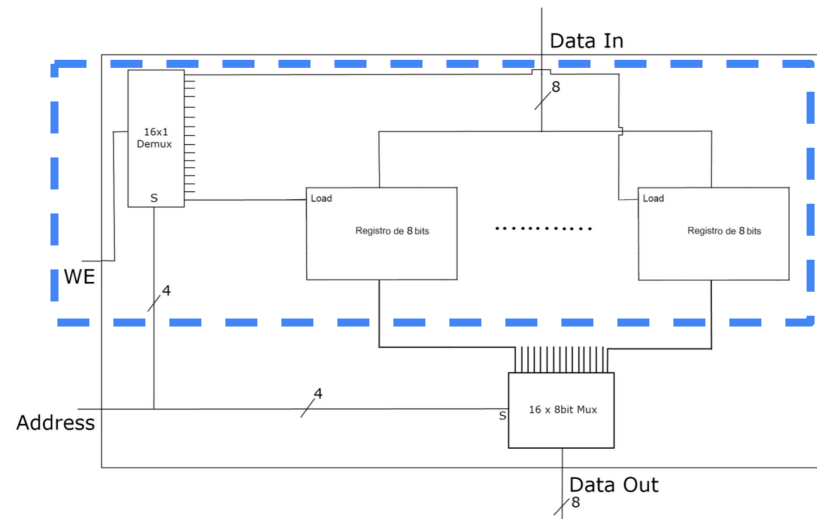
Tipos de memoria - RAM

Si estamos realizando una lectura, con el componente **16x8-bit Mux** seleccionamos **solo uno** de los valores de los registros de memoria internos a través del bus *Address* de 4 bits. Este resultado corresponderá al bus de salida *Data Out*.



Tipos de memoria - RAM

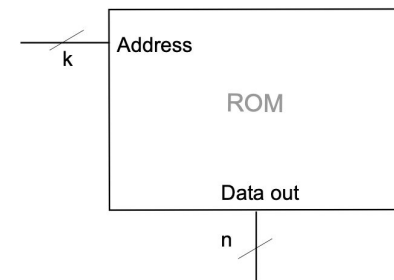
Si realizamos una escritura, con el componente **16x1-bit Demux** transmitimos la señal WE como *Load* de **solo** el registro de memoria seleccionado a través de la señal *Address* de 4 bits. Esto realizará la carga del bus de entrada *Data In* sobre el registro de memoria deseado. El bus de salida *Data Out* tendrá el mismo valor al escoger el mismo registro sobreescrito.



Tipos de memoria - ROM

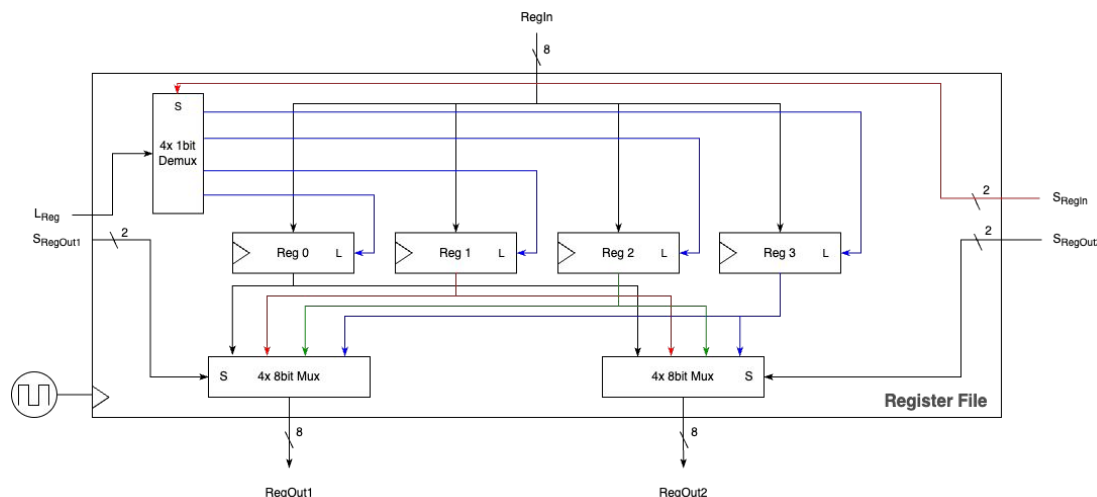
Memoria de solo lectura. En la práctica, se les llama así porque se “graban una vez” sin permitir sobreescritura (o con un costo considerable). Además, son **no volátiles**, por lo que retienen la información a pesar de cortar su suministro eléctrico.

Con k bits para el bus de direccionamiento y bus de salida de n bits, la ROM almacena 2^k palabras de memoria de n bits (con valores ya establecidos).



Tipos de memoria - *Register File*

Extensión de un registro. Su composición interna es similar al de una memoria RAM construida con registros, pero cuenta con dos salidas que se suelen utilizar como entradas de una ALU.



Si bien no lo usaremos en nuestra máquina programable, es un componente estándar dentro de la industria.

Memoria en la máquina programable - Variable

Hasta ahora, hemos visto distintos componentes para almacenar datos. ¿Cómo los usamos en nuestra máquina?

Cuando programamos, definimos **variables** que almacenan valores temporales que se actualizan según el flujo de código. Estas variables se almacenan en distintos componentes de memoria de nuestra máquina, pero en la próxima clase veremos cómo se lleva a cabo.

Variables - Tipos de dato

Las variables, ya sea de forma explícita (como en TypeScript) o implícita (como en Python), poseen un **tipo de dato**. Este es un atributo que deriva en varias características del dato almacenado, tales como: codificación, tamaño, valores y operaciones posibles.

Este atributo es de especial interés ya que **incide en la forma en que lo almacenamos en nuestra máquina**.

```
let variable: number = 27;
```

Identificador	Tipo de dato	Valor
---------------	--------------	-------

Definición de variable en TypeScript



Variables - Tipos de dato de interés

Tipo de dato	Codificación	Interpretación	#Bits de representación
char signed char	base 2 sin signo base 2 con signo en complemento de 2	carácter o entero positivo entero positivo o negativo	8 8
short unsigned short	base 2 con signo en complemento de 2 base 2 sin signo	entero positivo o negativo entero positivo	16 16
int unsigned int	base 2 con signo en complemento de 2 base 2 sin signo	entero positivo o negativo entero positivo	32 32
long unsigned long	base 2 con signo en complemento de 2 base 2 sin signo	entero positivo o negativo entero positivo	64 64
long long unsigned long long	base 2 con signo en complemento de 2 base 2 sin signo	entero positivo o negativo entero positivo	128 128
float double long double	punto flotante de precisión simple punto flotante de precisión doble punto flotante de precisión cuádruple	Racionales y casos especiales Racionales y casos especiales Racionales y casos especiales	32 64 128

Este es el tipo de dato de la máquina programable del curso.

Este es el tipo de dato de la máquina programable del proyecto.

* Más adelante veremos la representación de números de punto flotante.

Variables - *Endianness*

Supongamos que deseamos almacenar una variable cuya representación requiere de 16 bits, pero las palabras de memoria de nuestra unidad de almacenamiento son de 8 bits. ¿Cómo lo almacenamos?

El *endianness* determina el orden en el que se almacena la secuencia de un dato.

Un ejemplo a continuación.

Variables - Endianness

Supongamos que tenemos el dato “0000111101010101” y una memoria con palabras de 1 byte. Tenemos dos opciones:

Dirección de memoria (hexa)	Palabra almacenada (<i>big endian</i>)
0x00	00001111
0x01	01010101
0x02	-

Dirección de memoria (hexa)	Palabra almacenada (<i>little endian</i>)
0x00	01010101
0x01	00001111
0x02	-

Si la palabra más significativa se almacena en la dirección menor, el almacenamiento es de tipo **big endian**; si la palabra menos significativa se almacena en la dirección menor, el almacenamiento es de tipo **little endian**.

Memoria en la máquina programable - Arreglos

Un caso particular de las variables es que pueden ser definidas como **arreglos**. En este caso, ¿cómo podemos manejarlos en memoria?

Adicional a las características de una variable (tipo de dato, *endianness*), contamos con un **largo** y una **dirección de memoria de inicio**. Con estos atributos podemos **indexar** cada elemento para acceder a ellos en nuestra memoria.

A continuación, un ejemplo.

Arreglos - Almacenamiento

Supongamos que tenemos el siguiente arreglo definido (asumamos que cada número ocupa 1 byte).

Dirección de memoria (hexa)	Palabra almacenada
0x00	00000001
0x01	00000011
0x02	00000101
0x03	00000111
0x04	-

```
let variables: number[] = [1, 3, 5, 7];
```

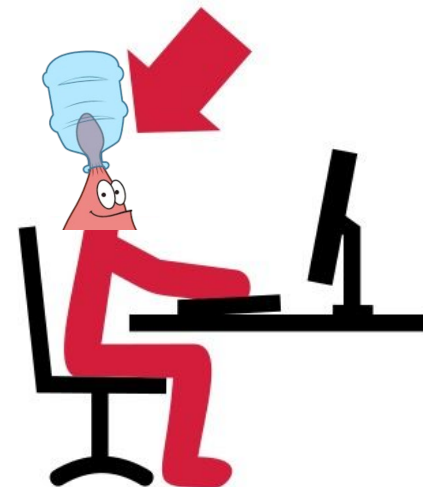
En este caso, la dirección de memoria de inicio sería **0x00** y debemos incrementar en una unidad este valor para acceder a cada elemento del arreglo.

La cantidad de unidades que sumamos a la dirección de memoria inicial **depende del tamaño del dato almacenado**.

Errores comunes

A continuación, veremos los errores más comunes que cometen en las interrogaciones de forma que puedan evitarlos.

PEBCAC



**Problem Exists Between
Chair and Computer**

Errores comunes

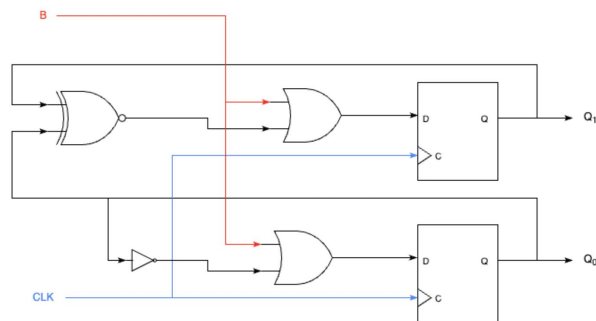
Diseño de contadores

- En ocasiones, se les solicita un contador con ciertas características. Por ejemplo:
 - Un contador incremental de 2 o 3 bits.
 - Un contador decremental de 2 o 3 bits.
 - Un contador con señal de reinicio
- En estos casos, se espera que diseñen el contador **con flip-flops**, **no que usen un contador para diseñarlo.**

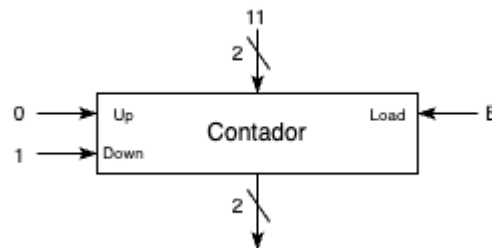
Errores comunes

Diseño de contadores

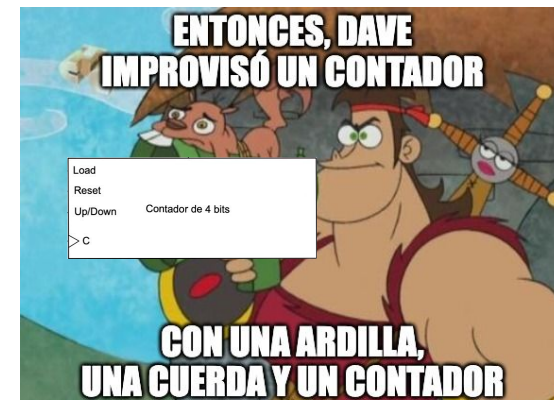
- **Ejemplo:** Diseñe un contador decremental de 2 bits que, al recibir una señal de entrada B igual a 1, reinicie su valor.



CORRECTO



INCORRECTO



Si tiene dudas, piense en esta imagen.

Ejercicios

Ahora, veremos algunos ejercicios.

Estos se basan en preguntas de tareas y pruebas de semestres anteriores, por lo que nos servirán de preparación para las evaluaciones.



Ejercicios

Construya el circuito del latch RS haciendo uso exclusivo de compuertas OR y NOT.

Interrogación 1, 2018-1

Ejercicios

Diseñe, utilizando todos los elementos de circuitos lógicos vistos en clases, un contador secuencial de 2 bits que se incrementa con cada flanco de subida de la señal de control.

Interrogación 1, 2013-1

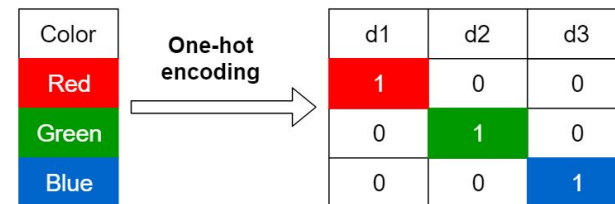
Ejercicios

Implemente mediante compuertas lógicas, elementos de control y latches, un flip-flop tipo D que funcione con flanco de bajada.

Interrogación 1, 2012-2

Ejercicios

Se desea tener un registro de estados de 4 bits de tipo *one-hot*, i.e. que siempre tiene un **único** bit igual a 1, no puede existir más de uno ni contener solo ceros. Modifique el diagrama de un registro para que este habilite su señal de carga L si, y solo si el valor de entrada cumple con ser *one-hot*.



Examen, 2023-1

Ejercicios

¿Cuántas direcciones tiene una memoria RAM de 4.5 KB que utiliza palabras de 3 bytes? (1 KB = 1024 bytes).

Interrogación 1, 2012-2

Ejercicios

Suponga que trabaja con un computador que utiliza datos de 2 bytes y que posee una memoria con palabras de 1 byte de almacenamiento. Describa, en este contexto, un mecanismo para determinar el *endianness* utilizado por el computador para el almacenamiento de datos.

Interrogación 1, 2016-2

Antes de terminar

¿Dudas?

¿Consultas?

¿Inquietudes?

¿Comentarios?





DCC
DEPARTAMENTO DE CIENCIA
DE LA COMPUTACIÓN

IIC2343

Arquitectura de Computadores

Clase 3 - Almacenamiento de Datos

Profesor: Germán Leandro Contreras Sagredo

Anexo - Resolución de ejercicios

¡Importante!

Estos ejercicios pueden tener más de un desarrollo correcto. Las respuestas a continuación no son más que soluciones que **no excluyen** otras alternativas igual de correctas.



Ejercicios - Respuesta

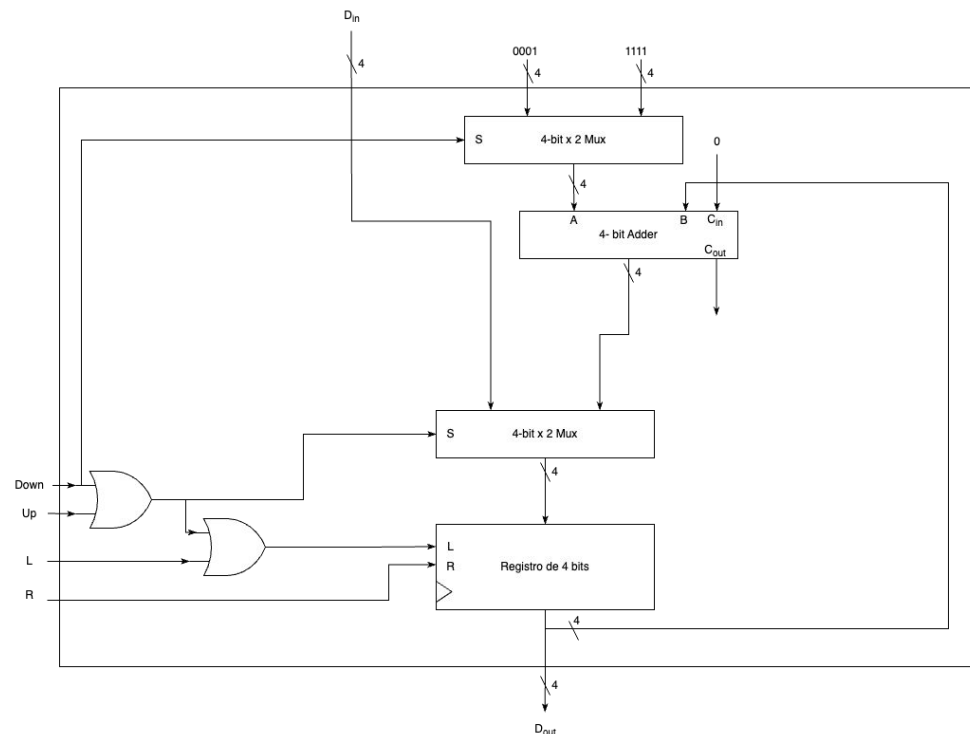
Modifique el diagrama interno del registro de memoria con *Load* y *Reset* de 4 bits para que incluya las señales *Up* y *Down*.

En este caso, nos basaremos plenamente en la construcción del registro de memoria con *Load* y *Reset* (y no a nivel de bit) para llevar a cabo la modificación.

Respuesta en la siguiente diapositiva.

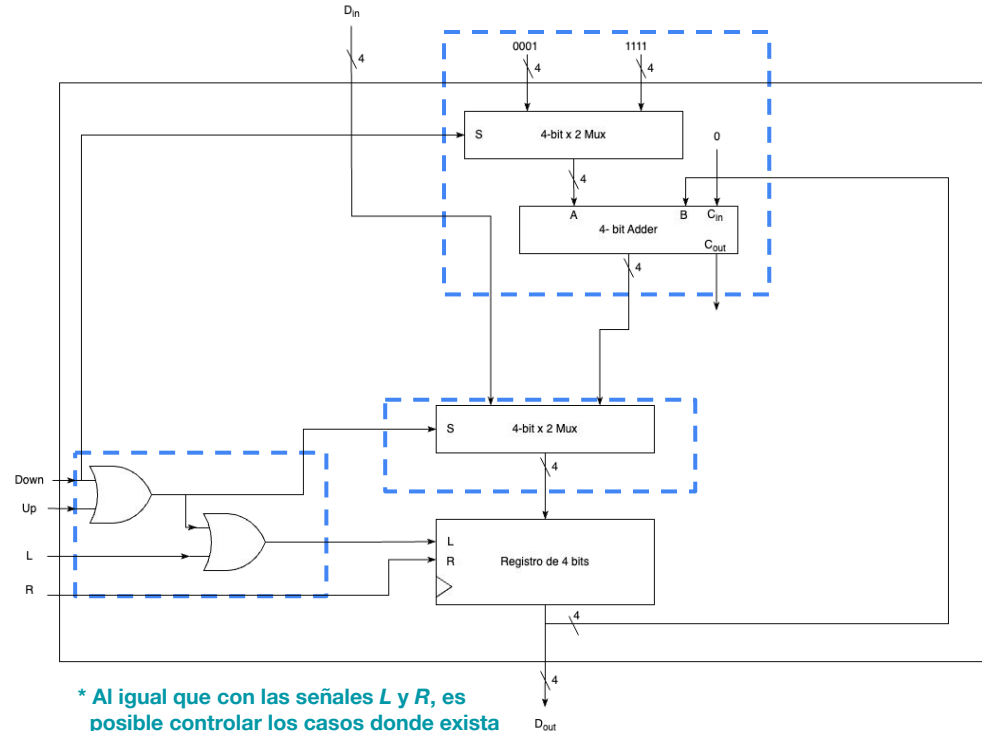
Ejercicios - Respuesta

En primer lugar, solo se lleva a cabo la carga sobre el registro si alguna de las siguientes señales está activa: *Load*, *Up*, *Down*. Por otra parte, la señal *R* sigue teniendo el mismo comportamiento.



Ejercicios - Respuesta

Si las señales *Up* o *Down* están activas, se seleccionará como ingreso de registro el resultado de la suma entre el estado previo y uno de dos valores: 0001 (1) o 1111 (complemento de 2 de 1). La selección de este número se realiza con la señal *Down*. Si ninguna de estas está activa, se selecciona el dato de entrada original (D_{in}) como el ingreso del registro interno.



* Al igual que con las señales *L* y *R*, es posible controlar los casos donde exista más de una señal activa para evitar comportamientos no esperados.

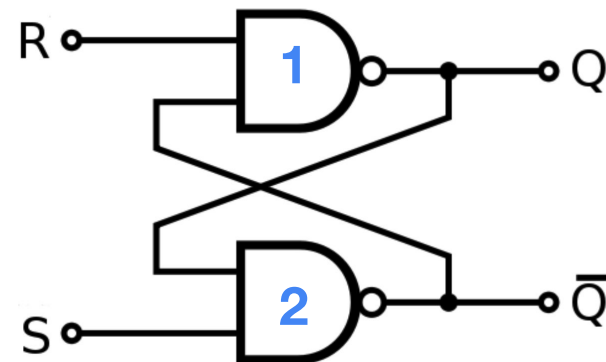
Ejercicios - Respuesta

Construya el circuito del latch RS haciendo uso exclusivo de compuertas OR y NOT.

La clave de este ejercicio es observar el circuito del latch RS con compuertas NAND y verlo según las fórmulas lógicas que lo definen.

NAND 1: $R \text{ NAND } \neg Q = \text{NOT}(R \text{ AND } \neg Q)$

NAND 2: $S \text{ NAND } Q = \text{NOT}(S \text{ AND } Q)$



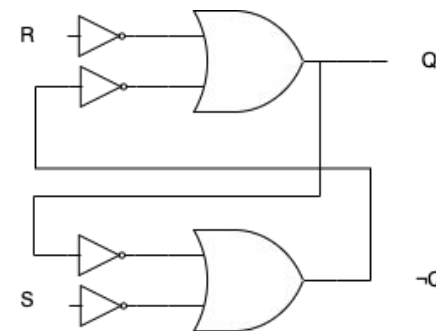
Ejercicios - Respuesta

Si aplicamos **Ley De Morgan**:

$$\text{NAND 1: } R \text{ NAND } \neg Q = \text{NOT}(R \text{ AND } \neg Q) = \neg R \text{ OR } Q$$

$$\text{NAND 2: } S \text{ NAND } Q = \text{NOT}(S \text{ AND } Q) = \neg S \text{ OR } \neg Q$$

Por lo tanto, basta con intercambiar las compuertas NAND por las combinaciones resultantes.



Ejercicios - Respuesta

Diseñe, utilizando todos los elementos de circuitos lógicos vistos en clases, un contador secuencial de 2 bits que se incrementa con cada flanco de subida de la señal de control.

Aquí lo esencial es ver cómo es la secuencia de números del contador y, en base a eso, definir la composición de cada bit.

La secuencia es: 00-01-10-11-00-01-10-11-00-... (hasta el infinito).

Ejercicios - Respuesta

Se puede observar que el bit menos significativo siempre alterna su valor entre 0 y 1 por cada iteración (*i.e.* por cada flanco de subida). Basta con que la entrada del flip-flop que almacena este dato sea su estado negado.

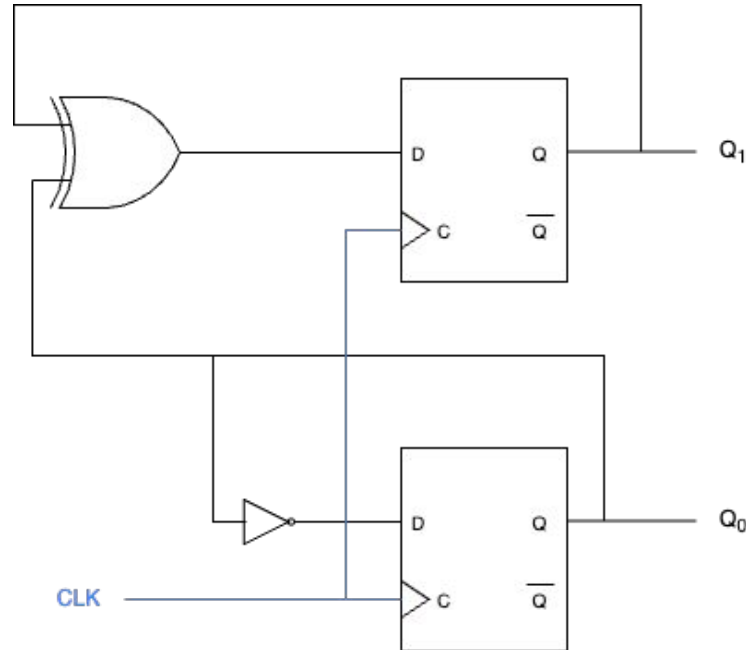
Para el bit más significativo, veamos la tabla de verdad respecto al número anterior de la secuencia.

Vemos que hay una compuerta precisa para representar este comportamiento: **XOR**.

$Q(t)_1$	$Q(t)_0$	$Q(t+1)_1$
0	0	0
0	1	1
1	0	1
1	1	0

Ejercicios - Respuesta

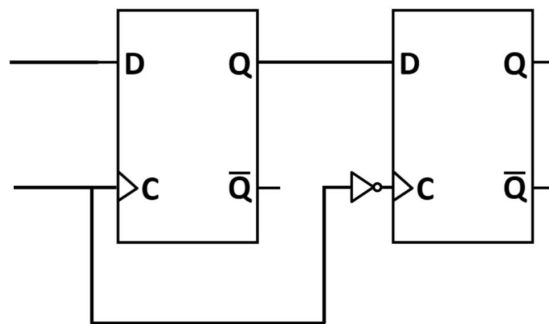
Finalmente, nuestro circuito quedará de la siguiente manera.



Ejercicios - Respuesta

Implemente mediante compuertas lógicas, elementos de control y latches, un flip-flop tipo D que funcione con flanco de bajada.

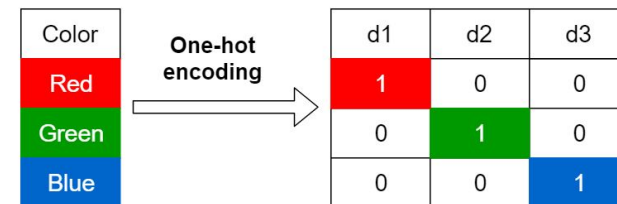
Simplemente tomamos como referencia la construcción con latches D en flanco de subida e invertimos el uso de la señal de control.



Ejercicios - Respuesta

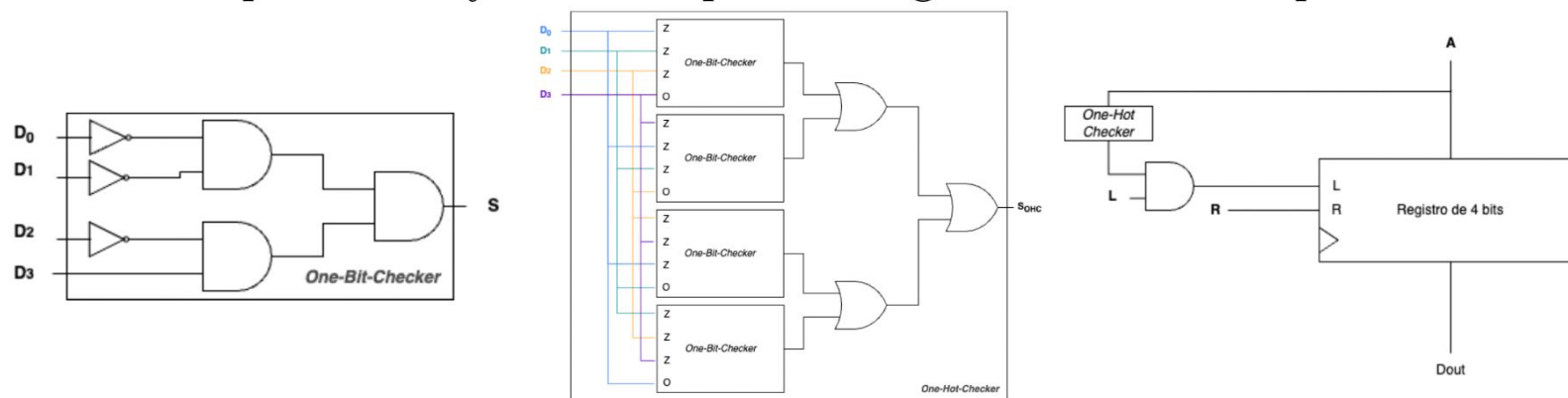
Se desea tener un registro de estados de 4 bits de tipo *one-hot*, i.e. que siempre tiene un **único** bit igual a 1, no puede existir más de uno ni contener solo ceros. Modifique el diagrama de un registro para que este habilite su señal de carga L si, y solo si el valor de entrada cumple con ser *one-hot*.

Solución en la siguiente diapositiva.



Ejercicios - Respuesta

Solución: Se diseñan dos componentes: *One-Bit-Checker* cuya salida es 1 solo si el bit más significativo de una entrada de 4 bits es el único igual a 1; y *One-Hot-Checker*, que hace uso de cuatro *One-Bit-Checkers* para determinar si una entrada de 4 bits cumple o con ser *one-hot* revisando si es igual a 1000, 0100, 0010 o 0001. Finalmente, se utiliza este último para recibir el valor de entrada del registro de 4 bits y se conecta su salida con una compuerta AND y la señal L para la carga sea efectiva solo para entradas *one-hot*.



Ejercicios - Respuesta

¿Cuántas direcciones tiene una memoria RAM de 4.5 KB que utiliza palabras de 3 bytes? (1 KB = 1024 bytes).

Sabemos que: Tamaño RAM = #Direcciones * Tamaño de palabra

Entonces, simplemente reemplazamos.

$$4.5 \text{ KB} = \# \text{Direcciones} * 3\text{B}$$

$$4.5 * 1024 \text{ B} = \# \text{ Direcciones} * 3\text{B}$$

$$\# \text{ Direcciones} = \mathbf{1536}$$

Ejercicios - Respuesta

Suponga que trabaja con un computador que utiliza datos de 2 bytes y que posee una memoria con palabras de 1 byte de almacenamiento. Describa, en este contexto, un mecanismo para determinar el *endianness* utilizado por el computador para el almacenamiento de datos.

Basta con almacenar el número 1 en memoria. Este, en hexadecimal, es equivalente a 0x01. Por el tamaño de memoria, una de las direcciones tendrá el valor 0x0 mientras que la otra tendrá el valor 0x01. Al leer la primer dirección de memoria de nuestro dato, podremos determinar su *endianness* si es que leemos un 1 o un 0.

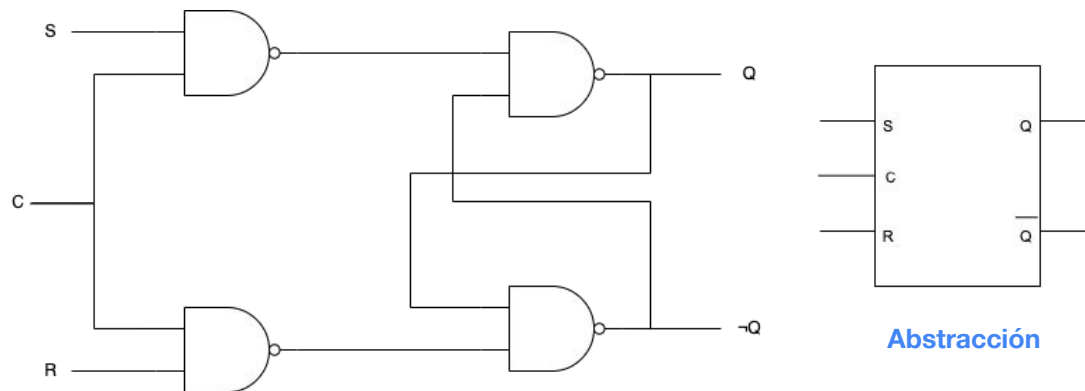
Anexo - Tipos de flip-flop y tipos de memoria

Si les interesa conocer más sobre los distintos tipos de flip-flop y de memoria, preste atención a las siguientes diapositivas.



Anexo - Flip-flop RS

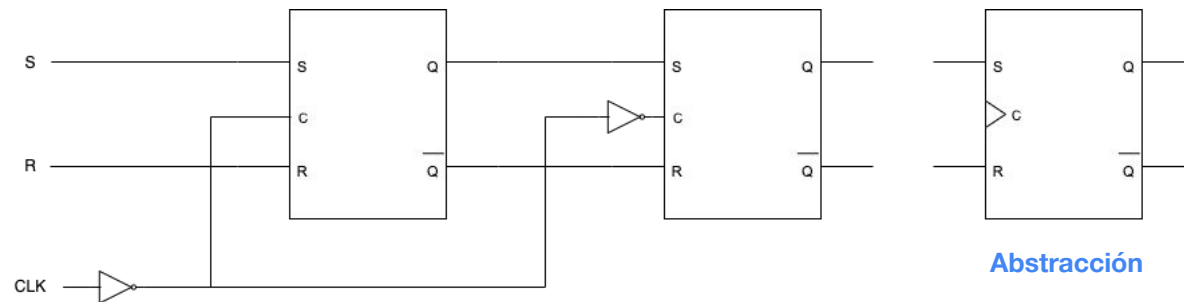
Similar al latch RS, pero con cambio de estado solo en flanco de subida. Para su construcción, primero se define el latch RS con señal de control C . Notar que por las compuertas NAND conectadas a la señal C , los *input* del latch RS se vuelven $\neg S$ y $\neg R$. Esto invierte las combinaciones que almacenan o invalidan el estado final.



C	S	R	$Q(t+1)$
0	0/1	0/1	$Q(t)$
1	0	0	$Q(t)$
1	0	1	0
1	1	0	1
1	1	1	-

Anexo - Flip-flop RS

Al igual que con el flip-flop D, se crea el flip-flop RS con una secuencia de latches, en este caso, del tipo RS con control C.

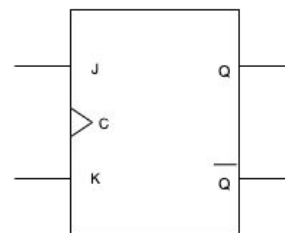
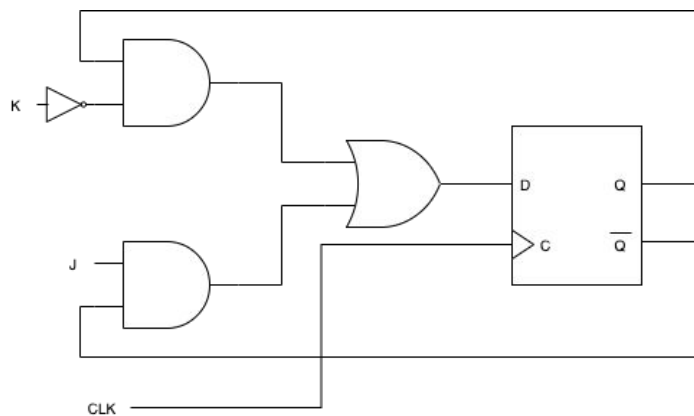


Abstracción

CLK	S	R	$Q(t+1)$
0/1/↓	0/1	0/1	$Q(t)$
↑	0	0	$Q(t)$
↑	0	1	0
↑	1	0	1
↑	1	1	-

Anexo - Flip-flop JK

Llamados así por su inventor, **J**ack **K**ilby. Funcionan de la misma forma que un flip-flop RS, pero en su caso no existe combinación inválida de señales de entrada, sino que esta combinación **invierte** el estado final.

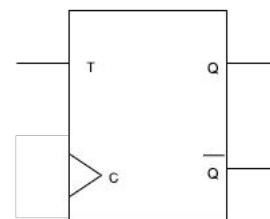
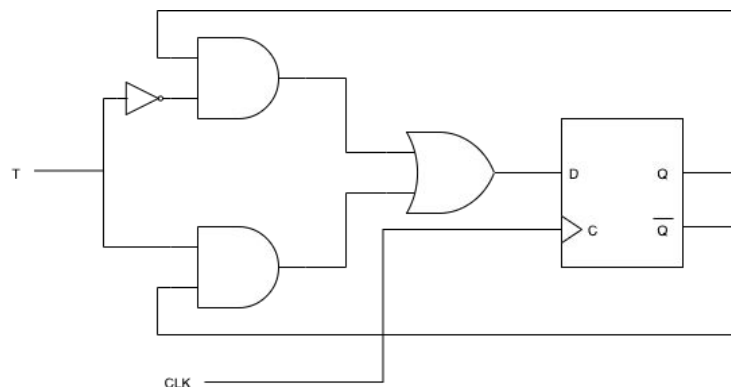


Abstracción

CLK	J	K	$Q(t+1)$
0/1/↓	0/1	0/1	$Q(t)$
↑	0	0	$Q(t)$
↑	0	1	0
↑	1	0	1
↑	1	1	$\neg Q(t)$

Anexo - Flip-flop T

Llamados así por el verbo *Toggle* (“alternar”). Cuando su señal T se activa, el estado del componente se alterna a su valor inverso.



Abstracción

CLK	T	$Q(t+1)$
0/1/↓	0/1	$Q(t)$
↑	0	$Q(t)$
↑	1	$\neg Q(t)$

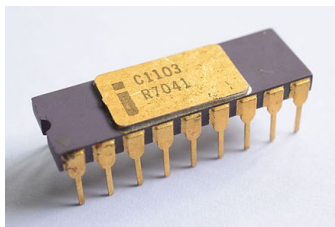
Anexo - Algunos comentarios adicionales

- Los flip-flop JK son los más utilizados por la flexibilidad que otorgan con la cantidad de combinaciones que poseen. No obstante, usaremos los flip-flop D por comodidad.
- En teoría, se pueden definir latches de tipo JK o T, pero no se usan en la práctica.
- Al tener dos latches en secuencia, estos suelen llamarse latch “maestro” y latch “esclavo” respectivamente.

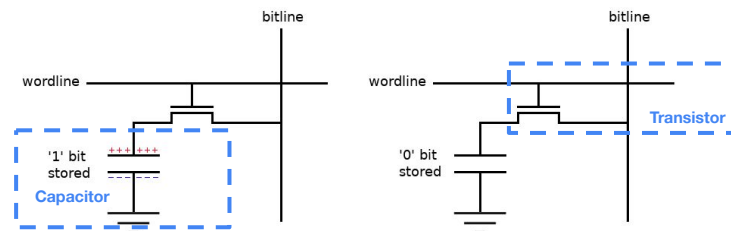


Anexo - Tipos de memoria RAM

Dynamic RAM o DRAM: Memoria RAM “dinámica” que almacena un bit a partir de un condensador y un transistor (celda de memoria). Son más económicas y fáciles de manufacturar, pero requieren un *memory refresh* (lectura y reescritura de las celdas de memoria cada ciertos milisegundos) ya que pierden su carga con facilidad.



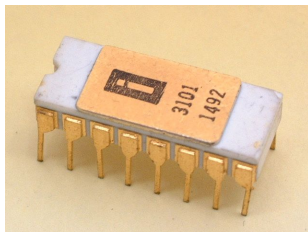
Intel 1103, primera DRAM de circuito integrado



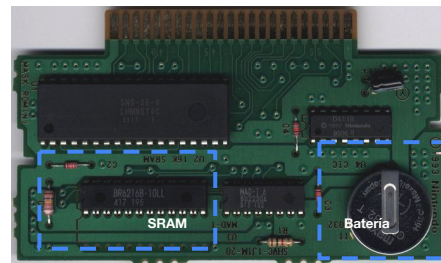
Esquema de almacenamiento de una celda de memoria con un transistor y capacitor.

Anexo - Tipos de memoria RAM

Static RAM o SRAM: Memoria RAM “estática” que almacena estados con flip-flops. Si bien necesitan más componentes internos y son más costosas, poseen un acceso de memoria más rápido que las DRAM. Por otra parte, no requieren de *memory refreshing*.



Intel 3101, primer producto oficial de la empresa.



Placa de un cartucho de Super Nintendo. Posee una SRAM para guardar el progreso y se mantienen los datos con la alimentación de una batería.

Formato de placa utilizado para los Donkey Kong Country. 🙌

Anexo - Tipos de memoria RAM

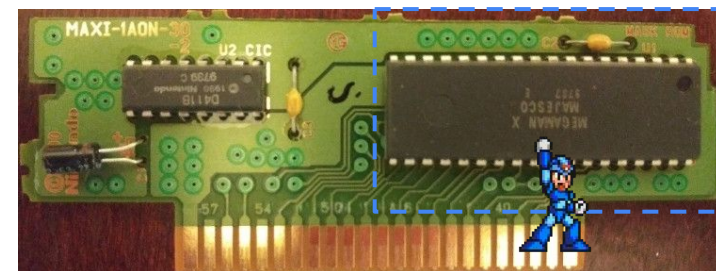
- Ambos tipos son volátiles, *i.e.* **necesitan alimentación eléctrica para funcionar**. Sin suministro, se pierden los datos almacenados.
- Por temas de costo, las DRAM se usan como la memoria principal de cómputo y las SRAM se usan como memoria caché.
- La más utilizada hoy en día es la DDR SDRAM: *Double Data Rate* (almacenamiento doble al hacerse en flancos de subida y bajada) *Synchronous* (sincronizada con el *clock* del sistema) DRAM.

Anexo - Tipos de memoria ROM

Las ROM iniciales fueron las Mask ROM o MROM, cuyo nombre nace de la “máscara” de metal utilizada para definir el patrón de datos grabado en el chip del componente.

Hoy en día otros tipos de ROM son más útiles al tener la conveniencia de poder modificar los datos escritos, tales como:

- PROM (*Programmable* ROM).
- EPROM (*Erasable* PROM).
- EEPROM (*Electrically Erasable* PROM).



Placa del cartucho de Super Nintendo del juego Mega Man X, incluyendo la MROM que contenían todas las placas de los juegos de esta consola.

Anexo - Tipos de memorias no volátiles

- **HDD:** *Hard Drive Disk* o unidad de disco duro.
 - Consiste en discos de metal con recubrimiento magnético que usan cabezales para modificar magnéticamente cada posición y, así, cada bit.
 - Almacena un mayor volumen de datos por un precio menor, pero con un considerable costo en términos de operaciones de lectura y escritura por sus movimientos mecánicos.



Anexo - Tipos de memorias no volátiles

■ Memoria flash

- Tipo de memoria que almacena sus datos de forma eléctrica con compuertas NAND o NOR.
- Usada para memorias USB, SD y SSD (*Solid State Drive*). Son más costosas que los HDD para un mismo tamaño, pero implican un menor costo en operaciones de lectura y escritura.



Anexo - Comparativa de tipos de memoria

Tipo de memoria	Tiempo de acceso promedio	Precio por GB promedio	Tamaño promedio
Registros	0.5 ns	\$\$\$\$\$\$\$\$\$	1 MB
RAM	50 ns	5 USD (2015)	8 GB
Disco de estado sólido	25000 ns	0.05 USD (2023)	512 GB
Disco magnético	5000000 ns	0.015 USD (2023)	1 TB

Esta referencia sirve para entender por qué los computadores no funcionan solo a base de registros de memoria o RAM.

Anexo - Almacenamiento de matrices

¿Cómo se almacenan los arreglos de dos dimensiones, *i.e.* las matrices? Podemos almacenar por convención de **filas** o **columnas**.

```
let variablesMatrix: number[][] = [
  [1, 3, 5],
  [2, 4, 6]
];
```

Dirección de memoria (hexa)	Palabra almacenada
0x00	00000001
0x01	00000011
0x02	00000101
0x03	00000010
0x04	00000100
0x05	00000110

Convención de filas

Dirección de memoria (hexa)	Palabra almacenada
0x00	00000001
0x01	00000010
0x02	00000011
0x03	00000100
0x04	00000101
0x05	00000110

Convención de columnas

Anexo - Direccionamiento de matrices

Tenemos distintas fórmulas de direccionamiento según la convención utilizada:

Convención por filas

$$\text{dir}(\text{matriz}[i,j]) = \text{dir}(\text{matriz}) + i * \text{sizeof}(\text{matriz}[i,j]) * \text{\#columnas} + j * \text{sizeof}(\text{matriz}[i,j])$$

Convención por columnas

$$\text{dir}(\text{matriz}[i,j]) = \text{dir}(\text{matriz}) + j * \text{sizeof}(\text{matriz}[i,j]) * \text{\#filas} + i * \text{sizeof}(\text{matriz}[i,j])$$

La cantidad de columnas o de filas representa el largo de cada subarreglo de la matriz, por lo que necesitamos considerarlos para un direccionamiento efectivo.

Anexo - Ejercicio de direccionamiento de matrices

Suponga que se tiene una matriz almacenada en la dirección de memoria 0x0A. Esta posee un total de 4 filas y 5 columnas. Si se sabe que en una dirección de memoria se puede almacenar 1 byte, y la matriz almacena en cada celda un dato de 2 bytes, ¿cuál es la dirección del dato que se encuentra en la tercera columna de la segunda fila de la matriz? Asuma que se utiliza la convención de filas.

Ayudantía 1, 2018-2, Germán Contreras

Anexo - Ejercicio de direccionamiento de matrices (Solución)

Haremos uso directo de la fórmula:

$$\text{dir}(\text{matriz}[i,j]) = \text{dir}(\text{matriz}) + i * \text{sizeof}(\text{matriz}[i,j]) * \#columns + j * \text{sizeof}(\text{matriz}[i,j])$$

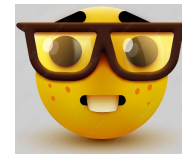
$$i = \text{segunda fila} = 1; j = \text{tercera columna} = 2; \#columns = 5; \text{sizeof}(\text{matriz}[i,j]) = 2$$

$$\text{dir}(\text{matriz}[1,2]) = 0x0A + 1 * 2 * 5 + 2 * 2 = 0x0A + 0x0E = 0x18$$

Como cada dato ocupa 2 bytes y tenemos una memoria de palabras de almacenamiento de 1 byte, las direcciones 0x18 y 0x19 corresponden al dato buscado.

Anexo - Videos complementarios

Por si desean profundizar más en los temas de esta clase.



- [Video interactivo de almacenamiento de 1 byte con condensadores.](#)
- [Resumen de las DRAM; funcionamiento de las celdas de memoria en una DRAM; ciclos de lectura y escritura en una DRAM.](#)
- [Funcionamiento DDR.](#)
- [Funcionamiento HDD; funcionamiento memoria flash.](#)
- [Bonus: Videos que tocan conceptos del curso con NES y SNES.](#)